IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2020

DIGITAL ELECTRONICS & COMPUTER ARCHITECTURE

Friday, 1 May 10:00 am

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions. Question One carries 40 marks. Question Two and Question Three carry 30 marks each.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : T.J.W. Clarke, Z. Durrani
Second Marker(s) : Z. Durrani, T.J.W. Clarke

© Imperial College London

Special instructions for invigilators: None

Information for candidates:

In the figures showing digital circuits, all components have, unless explicitly indicated otherwise, been drawn with their inputs on the left and their outputs on the right. All signals labelled with the same name are connected together. All circuits use positive logic. The least significant bit of a bus signal is labelled as bit 0, and the most significant bit with the highest integer number. Therefore the signal X[7:0] is an eight bit bus with X7 being the MSB and X0 the LSB. Furthermore, if A and B are registers, the notation A:B designates the registers concatenated with A more significant than B.

In questions involving circuit design, you may use any standard digital circuits that are not explicitly forbidden by the question provided that you fully specify their operation.

Marks may be deducted for unnecessarily complex designs unless you are explicitly instructed not to simplify your solution.

1. a) Simplify the following Boolean expressions using De Morgan's theorem and/or Boolean algebra.

(i) $f = \overline{(A + \overline{B})\,(\overline{A} + B)}$

[4]

(ii) $g = \overline{\overline{A}\overline{B}C}(B \oplus C)$

[4]

b) Simplify the following expression in (i) sum-of-products (SOP) and (ii) product-of-sums (POS) forms using Karnaugh maps. You may further assume that the term $A\bar{B}CD$ is a 'don't care'.

$$y = \overline{(C + D)} + \overline{A}\,C\,\overline{D} + A\,\overline{B}\,\overline{C} + \overline{A}\,\overline{B}\,C\,D + A\,C\,\overline{D}$$

[4+4]

c) Simplify the Boolean expression $A\,B + \overline{A}\,C + B\,C$ using Boolean algebra, such that the final expression is a sum of two product terms, each product term having only two variables.

[4]

d) Assuming that all numbers are 16 bits wide, complete the missing entries which are not shaded in the following table. Numbers should be in 2's complement form where needed. (No marks will be awarded for this question unless you show your working.)

Decimal	Hexadecimal	Unsigned Binary	BCD
7244	?		
		0000 0000 1101 0111	?
?	37FE		
-2417	?		

[8]

e) For the circuit shown in Figure 1.1, draw a truth table showing the outputs P and Q for all combinations of inputs A, B and C.

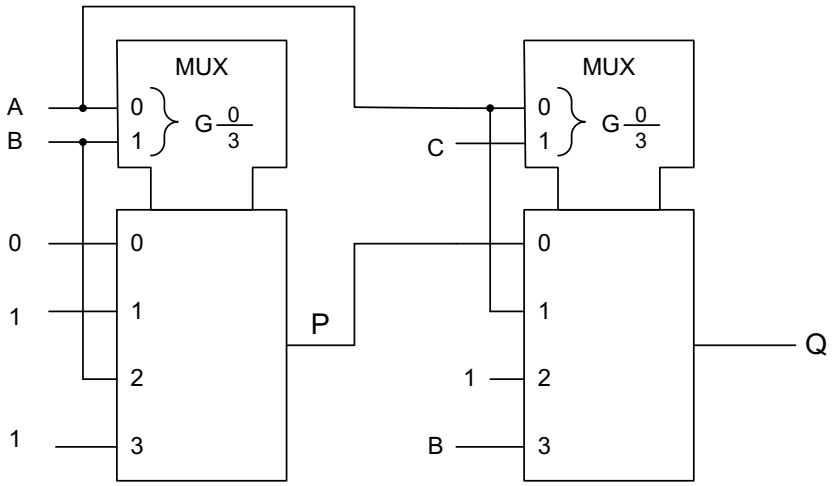


Figure 1.1

[6]

f) Figure 1.2 shows a finite state machine (FSM) implemented with a ROM that contains four 4-bit numbers. The ROM address lines are A[1:0] and the ROM data lines are D[3:0]. X and Y are output signals from the FSM. The contents of the ROM are shown in Figure 1.3. The flip-flops are initially in the reset state (i.e. Q0 = Q1 = '0').

Draw the (i) Moore state diagram, and (ii) the state transition table, for the FSM.

[3 + 3]

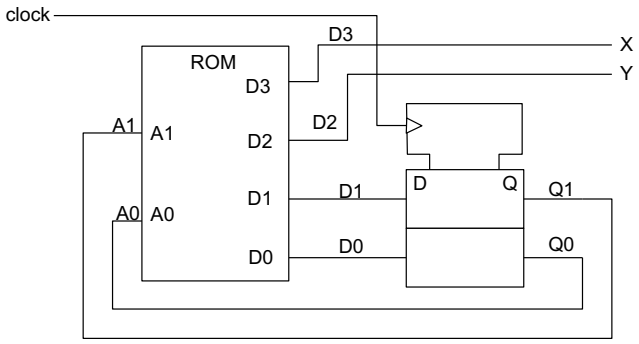


Figure 1.2

Address	Data
0	1101
1	1010
2	0111
3	0000

Figure 1.3

2. a) Figure 2.1 shows a sequence detector using a finite state machine (FSM), with input signal X and output signal Y. The FSM is required to detect the serial bit sequence 1101, where the MSB is to be detected first.

i) Draw a Mealy state diagram for the FSM.

[8]

ii) Draw the state transition table for the FSM.

[6]

iii) The FSM is to be implemented using D flip-flops and a combinational logic circuit. Write the output transition table for the D flip-flop. Hence, derive the Boolean expressions necessary for the combinational logic circuit.

[8]

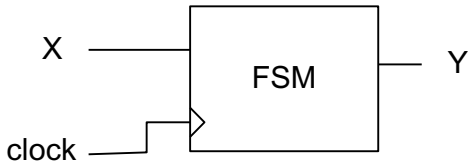


Figure 2.1

b) Figure 2.2 shows an arithmetic module COMP that compares the magnitude of two 4-bit unsigned numbers A[3:0] and B[3:0]. It produces a logic '1' on outputs H, E and L if A>B, A=B and A<B respectively. Derive the Boolean expressions for H, E and L. You do not need to simplify your Boolean equations.

[8]

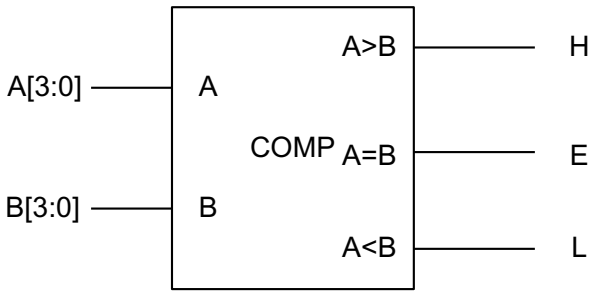


Figure 2.2

3. Figures 3.1 and 3.2 show a non-pipelined CPU and its RAM. The CPU state machine has 4 distinct states: F, E1, E2 and E3. The CPU instruction set is defined in Figure 3.3. You may assume that all CPU registers are initially 0.

From its start the CPU executes, in sequence, `LDI`, `LDA` and `LDN` instructions as determined by the program in Figure 3.4. Figure 3.5 shows how the CPU state changes during the first 10 cycles of this program executing.

a) Draw the state transition diagram of the CPU state machine. Indicate conditional state transitions as boolean expressions of `IRMUX` signals, assuming no instructions other than as in Figure 3.3. [6]

b) State the number of cycles delay between `Addr` changing, and the corresponding change in each of `Dout`, `IR`, `IRMUX`. Hence explain why the 4 DECODE signals from `IRMUX` could not come from `IR` or `Dout`. [6]

c) For each of the states E1, E2, E3 of the `LDN` (0x2100) instruction state the contents of Dout, and describe how these relate to the `LDN` operation. [6]

d) Name the memory addressing modes implemented by `LDA` and `LDN` instructions respectively, and state two advantages of immediate offset addressing, as used in the ARM CPU, over `LDA` and `LDN`. [4]

e) Figure 3.6 shows a 16 bit binary adder Σ with carry in and out `cin` and `cout` implementing single cycle two operand 16 bit arithmetic instructions. Two sequential cycles 0, 1 implement 32 bit subtraction: `R0:R1 := R0:R1 - R2:R3`.

i) State, with reasons, the values of `N` and `cin` in cycles 0 and 1. [6]

ii) State, using `X` for don't care, the values of `A` and `B` in each cycle. [2]

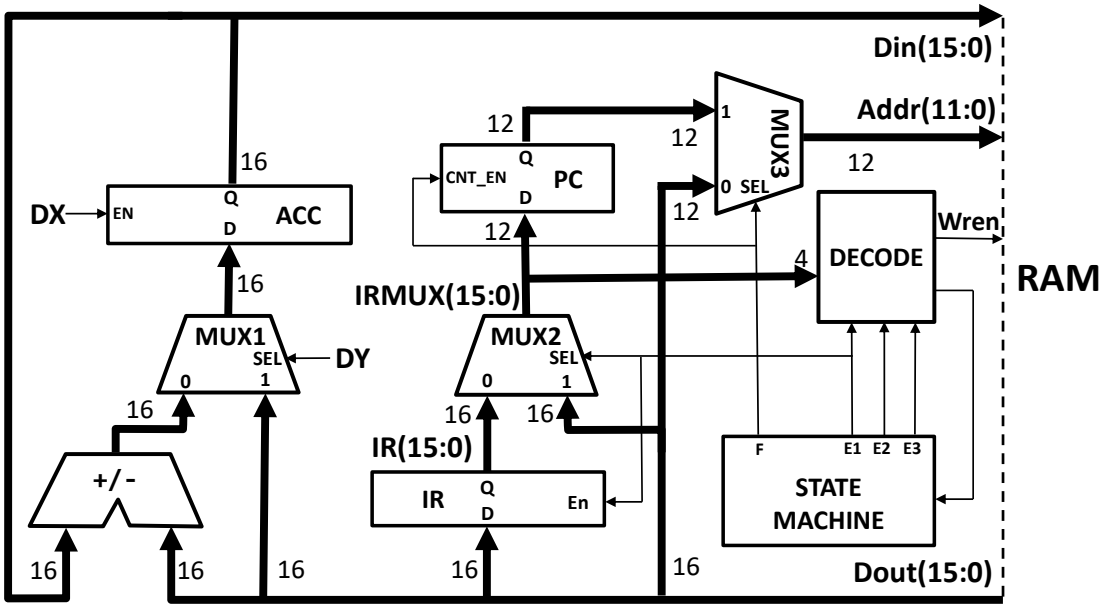


Figure 3.1: CPU Block Diagram

$\texttt{Wren}_N$	$\texttt{Addr}_N$	$\texttt{Din}_N$	$\texttt{Dout}_{N+1}$	Operation
0	a	X	$\texttt{Mem}_{16}[a]$	Read
1	a	d	X	$\texttt{Mem}_{16}[a]$:= d

Figure 3.2: RAM operation: S_N indicates signal S during cycle N

Mnemonic	IW(15:12)	Operation
LDI	0b0000	Acc := IW(11:0)
LDA	0b0001	Acc := $\texttt{Mem}_{16}$[IW(11:0)]
LDN	0b0010	Acc := $\texttt{Mem}_{16}$[$\texttt{Mem}_{16}$[IW(11:0)]]

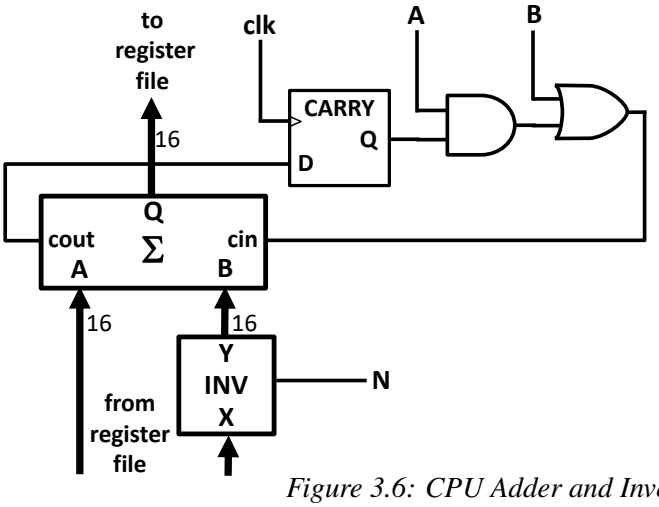
Figure 3.3: Instructions: IW(15:0) is the instruction word

Addr	$\texttt{Mem}_{16}$[Addr]	Assembler
0x000	0x0123	LDI 0x123
0x001	0x1100	LDA 0x100
0x002	0x2100	LDN 0x100
0x100	0x0101	
0x101	0x0246	

Figure 3.4: Initial RAM contents for program: all other locations are 0

Cycle	0	1	2	3	4	5	6	7	8	9
PC	0	1	1	2	2	2	3	3	3	3
State	F	E1	F	E1	E2	F	E1	E2	E3	F

Figure 3.5: States during first 10 cycles executing the program in Figure 3.4



INV block		
N	X(i)	Y(i)
0	x_i	x_i
1	x_i	$\overline{x_i}$

Figure 3.6: CPU Adder and Invert block